Android/iOS Screenshot Testing Strategy

1. Executive recommendation and quality objectives

News interfaces combine variable headlines, imagery, entitlement states and layouts. Frequent releases make exhaustive manual visual verification impractical. The objective is trustworthy PR feedback without an expensive or flaky automation bottleneck.

The brief requires a strategy for both platforms and a deterministic real-UI demonstration on at least one. My engineering objectives are economical reuse and less repetitive manual verification. Recommend shared scenario contracts, fast component screenshots where supported, a small Appium integration suite, and human review of intentional changes. Choose component tools after inspecting the production codebases; assume neither React Native, Compose nor SwiftUI.

2. Tooling architecture and trade-offs

Verified prototype: adapted Volksverpetzer React Native app; Appium 3.3.0/UiAutomator2 7.1.2; Python/pytest; `HomeScreen` exposing `ArticleCard`; separate Android session configuration and Pillow comparison; Pixel_10/API 37 emulator. Appium captures native rendered UI and offers reusable high-level Python scenarios with separate platform drivers. Its costs include device startup, slower execution, accessibility-tree dependencies and rendering variability.

Production layer	Choice and trade-off
Android components	Evaluate Paparazzi for isolated supported Views/Compose rendering; Roborazzi when Robolectric integration is useful. JVM execution avoids emulator startup, but renderer compatibility and integrated behavior need separate validation.
iOS components	Evaluate swift-snapshot-testing for UIKit views/controllers and appropriately hosted SwiftUI components. Local isolation supports many states; iOS rendering still needs a consistent simulator/toolchain.
Integrated journeys	Appium UiAutomator2 on Android; proposed XCUITest adapter on iOS. Verify real composition/navigation with fewer cases because device execution costs more.

Compose snapshot tools do not directly render arbitrary React Native components. Use the lowest-cost compatible layer providing the required confidence; avoid duplicating identical assertions. Share fixture schemas, scenario intent, comparison/reporting policy and CI templates. Keep locators, launch/reset, fixture adapters and reference PNGs platform-specific.

3. Deterministic mock-data architecture

Implemented: `__DEV__` plus `EXPO_PUBLIC_QA_SCENARIO=long_headline` selects a fixture fetcher. Synthetic article data passes through the existing WordPress mapping into real `Feed`, `GenericPost` and `ArticlePost` components. A bundled image and fixed headline, metadata and summary control the card. Appium waits for visible `qa-article-card-ready`, exposed after image loading, and checks the fixture-title prefix. Appium supplies automation; the app supplies mocks. This does not prove complete app-wide network isolation.

Proposed production seam: version a shared fixture schema and scenario identifiers alongside tests, jointly maintained by development and QA, with platform-specific adapters. Inject at the repository/data-source boundary through dependency injection in a test variant. Where unavailable, redirect the API client to a controlled local mock server with fixed routes and assets. Isolate fixture selection from production; verify release builds cannot enable it.

Each scenario specifies bundled images, frozen time/timezone, locale, theme, font files/font scale, animation policy and session/entitlement state. Reset caches and sessions between cases; await fonts, images and asynchronous layout explicitly. Block unexpected external requests in test mode and report them. Keep production networking unchanged.

4. Golden image management and review

Partition references by platform, device size, OS/renderer and display configuration. Commit only approved PNGs with their fixture/profile versions; consider LFS if repository growth justifies it. Publish actual, expected, diff and JSON metrics together. A missing baseline requires approval and a dimensions mismatch fails without resizing. CI never regenerates or accepts goldens automatically; unexpected differences fail. An intentional update is proposed as an explicit PR with rationale and comparison evidence. Protect reference directories with required reviewer ownership. After signoff, rerun verification against the proposed image before merging; retain history of the previous reference.

For intentional changes, the developer explains the design change and submits a candidate golden with comparison evidence. A feature reviewer and designer approve the visual result; QA checks fixture/profile consistency. Promotion is an explicit PR change, followed by verification; retain the previous reference in version history.

Verified baseline: `qa_visual/baselines/android/pixel10-api37/long_headline.png`. The native image is **1028 × 1237**; two captures were byte-identical. Local pytest returned **1 passed, exit 0**, with **0 changed pixels**. The comparator counts a pixel when any RGB channel difference exceeds **12**, allowing at most **0.2%** changed pixels. These values describe this profile, not universal defaults; calibrate others against repeatability and known defects.

Boundary inspection mattered: the initial inner element omitted **105 px** of the outer card. The corrected parent capture checks containment and includes the lower padding/corners. Full-screen context remains diagnostic, excluding its changing clock from comparison.

5. CI/CD and Scrum integration

Proposed PR workflow; CI is not implemented:

- Build the appropriate test variant with pinned dependencies.
- Select versioned deterministic fixtures and reset application state.
- Run affected-component screenshots where supported.
- Execute a focused device smoke suite for critical journeys.
- Compare with reviewed platform/profile references.
- Fail unexpected changes; publish actual/golden/diff/metrics and logs even on failure.
- Require human review for baseline updates; rerun verification after approval.

Provision Android runners with emulator acceleration and sufficient memory. For iOS, use macOS, compatible Xcode/simulator versions and the Appium XCUITest driver; install the test build, inject the shared scenario through an iOS adapter, then capture against separate iOS goldens. A device cloud is optional for broader hardware coverage.

During refinement, developers, QA, designers and product owners agree critical states and expected appearance. Developers maintain component tests; QA owns integration diagnostics and selection rules; designers review intended visuals; product owners resolve acceptance ambiguity. Keep PR checks focused using component ownership/change maps; run themes, font scales, device sizes, OS versions, offline/errors and premium states nightly or before release.

6. Flakiness, scaling and rollout

Pin emulator/simulator images, fonts, assets and rendering settings. Use explicit readiness, never arbitrary sleeps. Capture screenshots, accessibility trees and logs; distinguish infrastructure/readiness failures from genuine UI regressions. Permit one logged retry for diagnosed transient infrastructure errors, preserving both attempts; never retry a pixel mismatch until green. Quarantine persistent flakes with an owner and repair deadline.

Pilot a few high-risk components and one device profile: long headlines, breaking banners and premium badges. Establish baseline ownership and repeatability before making checks blocking. Introduce focused PR gates, then an iOS pilot and additional fixtures; expand nightly/release coverage according to risk and measured stability. Pair developers and QA engineers on fixture seams, diagnosis and review guidelines; document examples for other teams.

Measure PR feedback latency, false-positive rate, escaped visual defects, CI cost, flaky-test frequency and manual verification effort. Establish initial measurements before claiming improvement; use them to adjust coverage and runner capacity.

7. Demonstrated scope and limitations

Implemented and verified: one Android article-card scenario in the real app, native component screenshot, reviewed golden, repeatability and passing local comparison. It proves the fixture-to-render-to-screenshot-to-comparison path, not broad component or device coverage. See the runner guide and execution record.

Not implemented or verified: native iOS execution, completed CI pipeline, wider scenario/device coverage, deliberate failing-regression evidence, or Git commit/push. The existing diff artifact accompanies a passing comparison. This representative article card is not a complete production test matrix; additional representative components remain a submission gap.